



Probabilistic Machine Learning

Linear Algebra(7.4~), Optimization(8)

김무진 / '26.03.05



Computational Data Science LAB

CONTENTS

1. Linear Transformation (선형변환)
2. Eigen Value Decomposition (고윳값 분해)
3. Singular Value Decomposition (특잇값 분해)
4. Hessian Matrix (헤세 행렬)
5. Optimization (최적화)
6. Constrained Optimization (제약 최적화)

01 | Linear Transformation

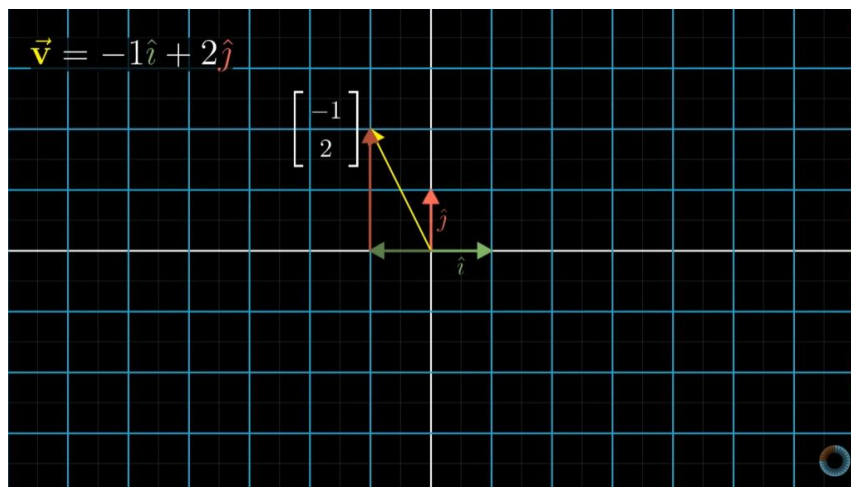
선형변환

- 선형변환

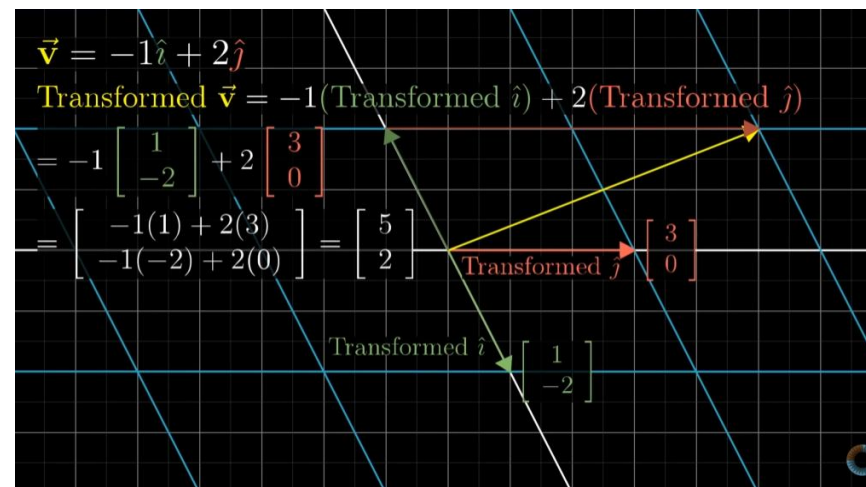
- ✓ 모든 $v, w \in V$ 에 대해 $f(v + w) = f(v) + f(w)$ 이고, $f(av) = af(v)$ 인 임의의 함수 $f: V \rightarrow W$
- ✓ 회전, **전단**(원을 타원으로 변환하는 과정) 등 존재

- 선형변환의 기하적 의미

✓



✓ 행렬 - 벡터 곱 $\begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = x \begin{pmatrix} a \\ c \end{pmatrix} + y \begin{pmatrix} b \\ d \end{pmatrix} = \begin{pmatrix} ax + by \\ cx + dy \end{pmatrix}$



(참고 : [3Blue1Brown - YouTube](#))

01 | Linear Transformation

행렬-행렬 곱의 관점

- 행렬-행렬 곱을 바라보는 관점

- ✓ $\begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} e & f \\ g & h \end{pmatrix} = \begin{pmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{pmatrix}$

- ✓ 내적 관점

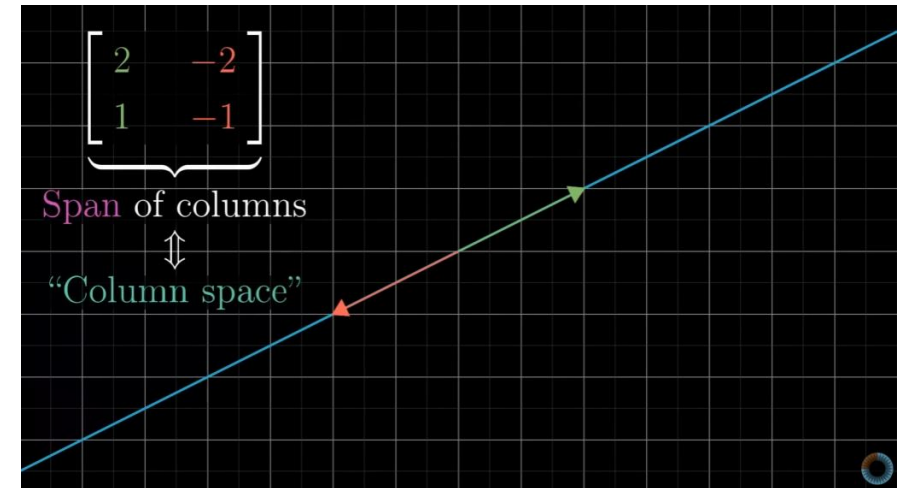
- $\begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} e & f \\ g & h \end{pmatrix} = \begin{pmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{pmatrix}$

- ✓ Rank-1 행렬들의 합

- $\begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} e & f \\ g & h \end{pmatrix} = \begin{pmatrix} ae & af \\ ce & cf \end{pmatrix} + \begin{pmatrix} bg & bh \\ dg & dh \end{pmatrix}$
 - UV^T 의 합

- Rank

- ✓ 선형변환 후의 출력의 차원 수
- ✓ 열공간(변환된 기저벡터들의 span)의 차원 수
- ✓ 2X2 행렬의 경우 가능한 최대 rank는 2
- ✓ 2X2 행렬의 rank가 1인 경우는 공간을 붕괴시킨 것



02 | Eigen Value Decomposition (고윳값 분해)

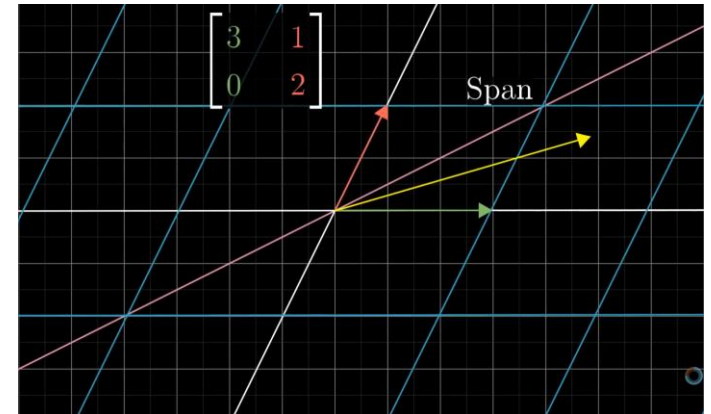
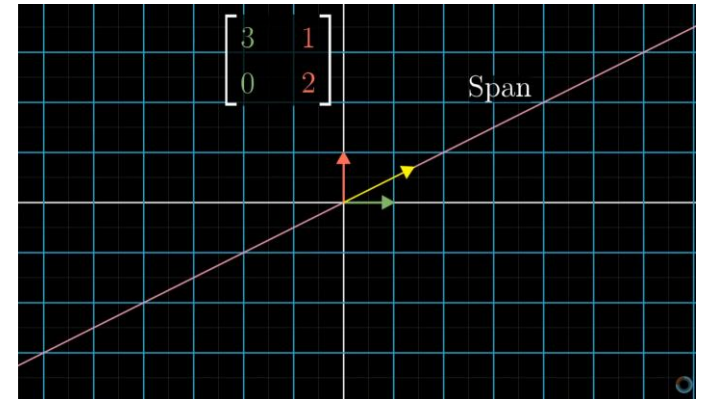
EVD

- 고윳값

- ✓ 선형변환 시에 대부분의 벡터는 변환의 과정에서 자신의 span을 벗어 날 것
- ✓ **정방행렬** $A \in R^{n \times n}$ 가 주어졌을 때, $Au = \lambda u, u \neq 0$
- ✓ 고윳값은 특성방정식 $\det(\lambda I - A) = 0$ 의 해

- 고윳값 분해

- ✓ $AU = U\Lambda, U = [u_1, u_2, \dots, u_n], \Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$
- ✓ 만약, 행렬 A 의 고유벡터들이 서로 선형 독립이라면, 행렬 U 는 역행렬이 존재
- ✓ $A = U\Lambda U^{-1}$
- ✓ 왜 대각화가 필요한가?
 - 행렬의 거듭제곱
 - 데이터의 구조 파악 $\rightarrow U$ 는 데이터의 주요 축(방향)을, Λ 는 그 축으로의 분산을 나타내어 PCA와 같은 차원 축소의 기반
 - 공분산 행렬을 고윳값 분해 했을 때, 고유벡터가 가중치 행렬, 고윳값의 대각행렬이 주성분의 공분산 행렬이 됨

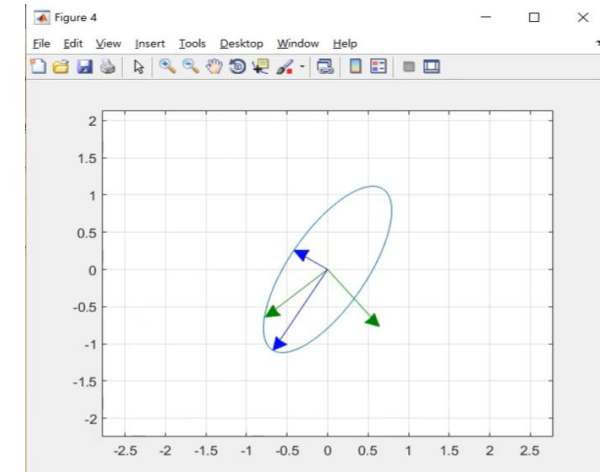
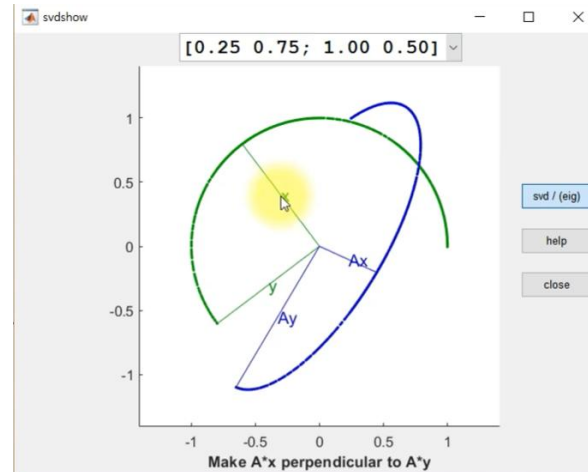
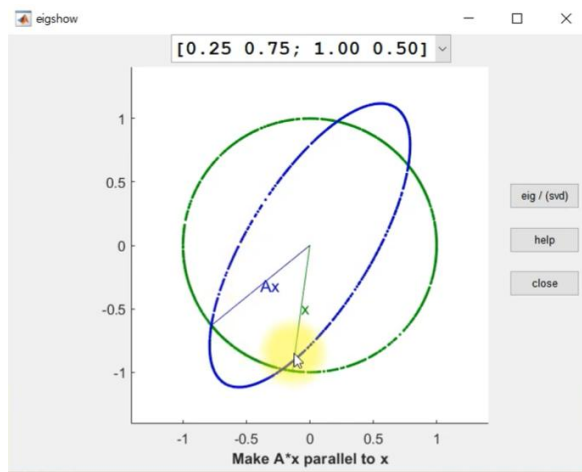


03 | Singular Value Decomposition (특잇값 분해)

SVD의 기하적 접근

- SVD의 기하적 접근

- ✓ 원을 구성하는 수많은 직교하는 두 벡터에 대해 타원으로 변환하는 과정에서 여전히 직교인 벡터(장축과 단축)를 찾기



(참고: https://youtu.be/cq5qLYtnLoY?si=eSvPqsZCgD8p_hHy)

- ✓ 즉, 행렬이라는 선형변환을 타원의 장축과 단축을 기준으로 회전 → 스케일링 → 회전의 3단계로 보고자 함

03 | Singular Value Decomposition (특잇값 분해)

SVD의 수식적 접근(1)

- 정규 직교행렬

- ✓ 길이가 1인 행벡터와 열벡터가 유클리드 공간의 정규 직교 기저를 이루는 실수 행렬 (위키피디아)

- ✓ 실수 $n \times n$ 행렬 Q 에 대하여, $QQ^T = Q^TQ = I_n$, $Q^{-1} = Q^T$

- ✓ $Q = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \rightarrow \det(Q) = +1$ (회전)

- ✓ $W = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \rightarrow \det(W) = -1$ (반사)

- SVD의 수식적 접근

- ✓ V : 행과 열이 정규직교인 $n \times n$ 행렬 ($V^TV = VV^T = I_n$)

- ✓ Σ : 주대각에 특잇값 $\sigma_i \geq 0$ 을 가지며 행렬의 나머지는 0으로 채워진 $m \times n$ 행렬

- ✓ U : 열이 정규 직교인 $m \times m$ 행렬

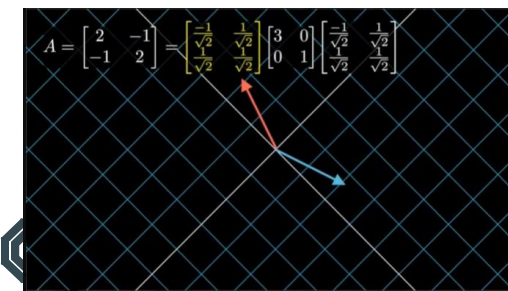
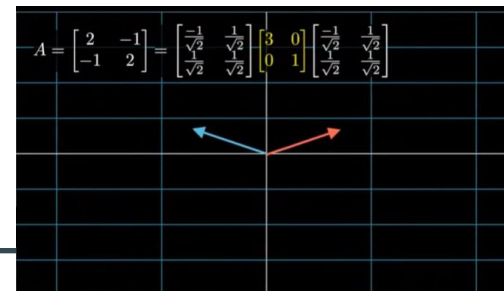
- ✓ $AV = U\Sigma$

- ✓ $A = U\Sigma V^{-1} = U\Sigma V^T$

- 대각행렬

- ✓ $\Sigma = \begin{pmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \end{pmatrix}$

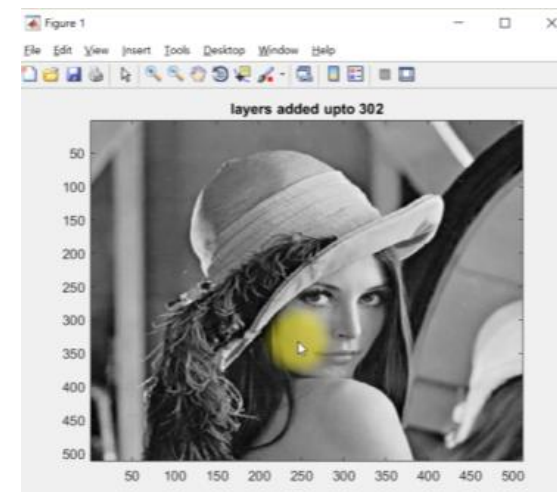
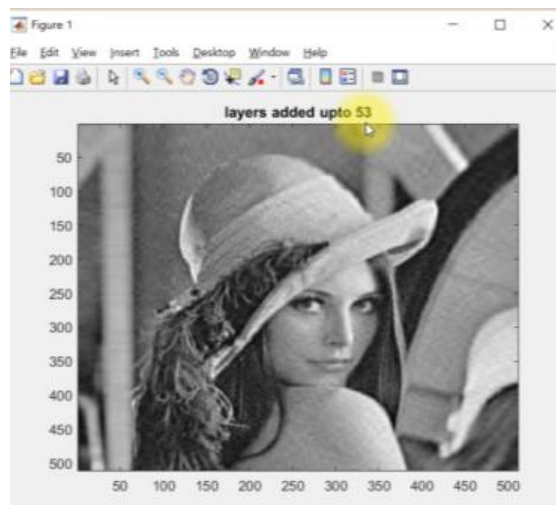
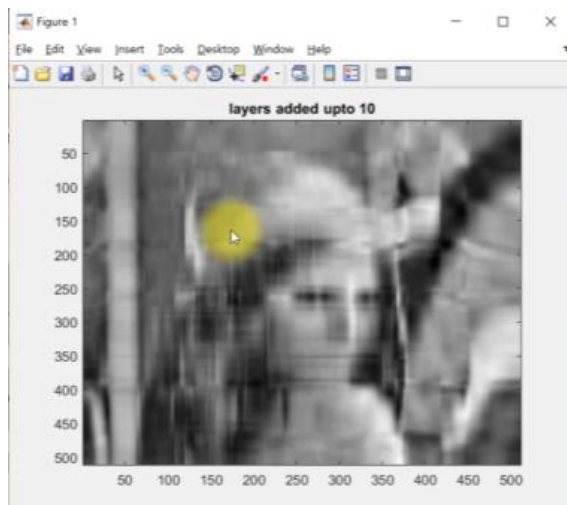
- ✓ $\begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} (3 \times x) + (0 \times y) \\ (0 \times x) + (1 \times y) \end{pmatrix} = \begin{pmatrix} 3x \\ y \end{pmatrix}$



03 | Singular Value Decomposition (특잇값 분해)

SVD의 수식적 접근(2)

- Rank-1 행렬들의 합
 - ✓ $U\Sigma V^T = \sigma_1(u_1 v_1^T) + \dots + \sigma_n(u_n v_n^T) \rightarrow$ 정보량의 크기에 따라 행렬 A 를 여러 Layer로 나누게 됨
 - ✓ Numpy는 언제나 특잇값을 내림차순으로 반환함
 - ✓ 즉, 가장 중요한 앞 쪽의 몇 개 덩어리($\sigma_1, \sigma_2 \dots$)만 더해도 원본 행렬의 핵심 정보를 거의 완벽하게 복원할 수 있음
 - ✓ 차원 축소나 이미지 압축의 원리



(참고: https://youtu.be/cq5qYtnLoY?si=eSvPqsZCgD8p_hHy)

04 | Hessian Matrix (헤세 행렬)

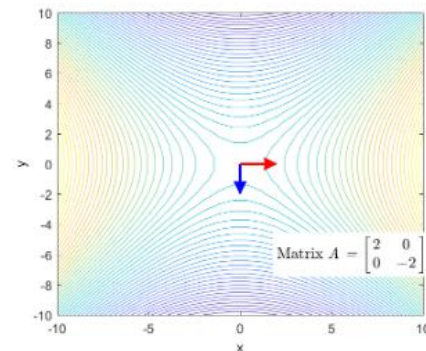
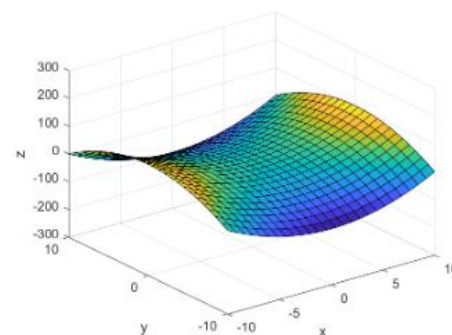
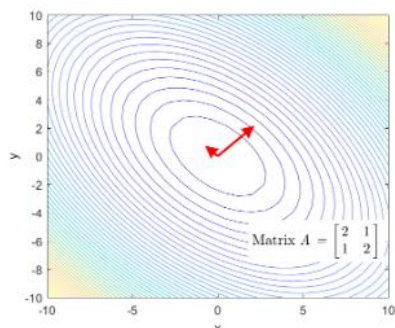
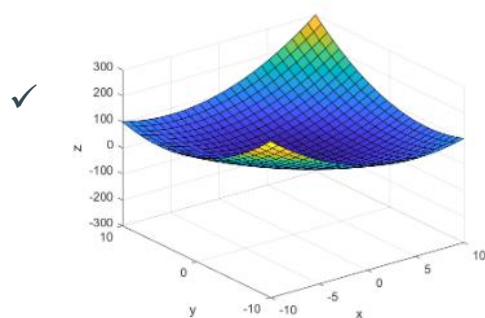
Hessian Matrix(헤세 행렬)의 기하적 의미

- Hessian Matrix(헤세 행렬)

- ✓ 두 번 미분 가능한 함수 $f: \mathbb{R} \rightarrow \mathbb{R}$ 에 대해 헤세 행렬은 다음과 같이 정의됨

- ✓
$$\mathbf{H}_f = \frac{\partial^2 f}{\partial \mathbf{x}^2} = \nabla^2 f = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{pmatrix}$$

- Hessian Matrix의 기하적 의미



(참고 : <https://angeloyeo.github.io/2020/06/17/Hessian.html>)

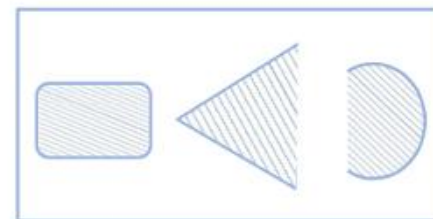
- ✓ 즉, Hessian Matrix를 이용하면 함수의 특정 위치가 위로 볼록한 지, 아래로 볼록한 지, 안장점인 지 알 수 있음

05 | Optimization (최적화)

볼록 집합과 볼록 함수

- Convex Set (볼록 집합)

- ✓ 임의의 $x, x' \in S$ 에 대해 다음과 같다면, S 는 볼록 집합
- ✓ $\lambda x + (1 - \lambda)x' \in S, \forall \lambda \in [0, 1]$



Convex

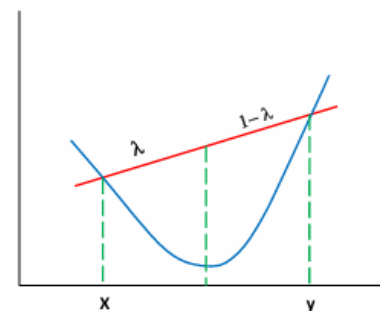


Not Convex

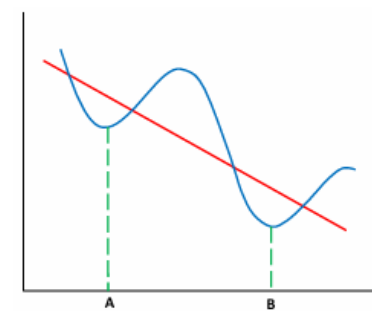
Figure 8.3: Illustration of some convex and non-convex sets.

- Convex Function (볼록 함수)

- ✓ 함수 f 의 그래프 윗부분이 볼록 집합을 정의한다면 이는 볼록 함수라 말함
- ✓ 목적 함수가 볼록 함수인 경우, 국소 최솟값은 또한 전역 최솟값
- ✓ 그러므로 많은 모델이 목적 함수가 볼록이도록 설계 되어있음
- ✓ $f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$



(a)



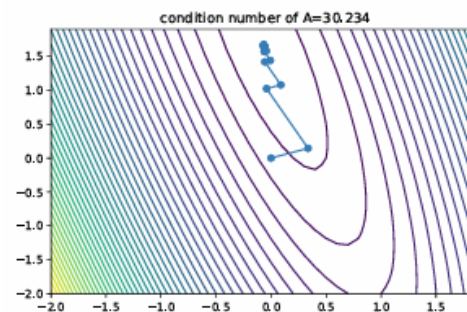
(b)

05 | Optimization(최적화)

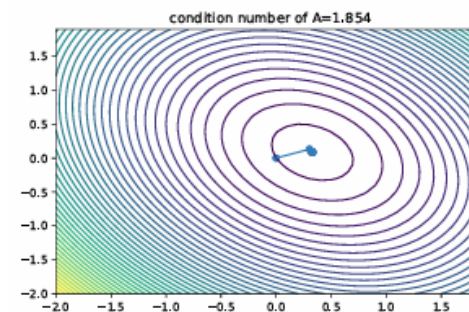
일계법과 이계법

- 일계법

- ✓ 목적 함수의 일계 도함수를 활용하는 반복적인 최적화 방법
- ✓ 어떤 방향이 '내리막'을 가리키는지 계산하지만 곡률 정보는 무시
- ✓ $\theta_{t+1} = \theta_t - \eta_t d_t$
- ✓ η_t 는 단계 크기 또는 학습률



(a)



(b)

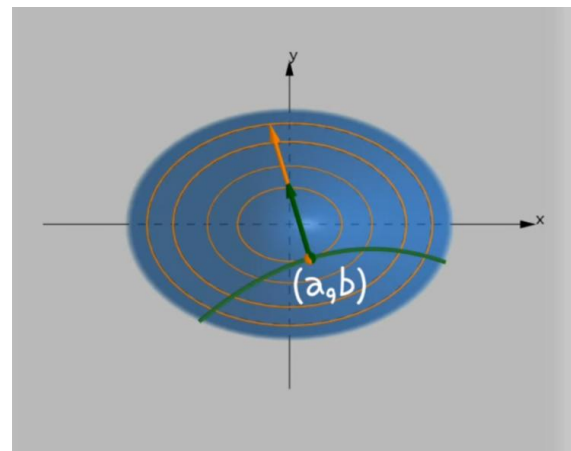
- 이계법(뉴턴법)

- ✓ $\theta_{t+1} = \theta_t - \eta_t \mathbf{H}_t^{-1} g_t$
- ✓ 역행렬 $\mathbf{H}^{-1}$ 가 국소적인 곡률의 비틀림을 되돌려서 경사 하강법보다 빠르게 최적값에 도달함

06 | Constrained Optimization (제약 최적화)

Lagrange Multipliers (라그랑주 승수)

- Lagrange Multipliers (라그랑주 승수)
 - ✓ 등식 제약화 최적화 문제를 풀고자 함
 - ✓ $\theta^* = \operatorname{argmin}_{\theta \in C} L(\theta)$, $h(\theta)=0$ 이 있다고 가정
 - ✓ $\nabla L(\theta^*) = \lambda \nabla h(\theta^*)$
 - ✓ $L(\theta, \lambda) \triangleq L(\theta) + \lambda h(\theta)$
 - ✓ $\nabla_{\theta, \lambda} L(\theta, \lambda) = 0$
 - ✓ $\lambda \nabla_{\theta} h(\theta) = \nabla_{\theta} L(\theta)$, $h(\theta) = 0$



$$f(x, y) = 4 - x^2 - 2y^2$$
$$g(x, y) = 2(x - 1)^2 - 10y + 3 = 0$$

$$g(a, b) = 0$$

$$\nabla f(a, b) = \lambda \nabla g(a, b)$$

참고 : (https://youtu.be/lmD9p6J_-TA?si=hCzSgATUfTEDXt1g)

06 | Constrained Optimization (제약 최적화)

일반화와 KKT Conditions

- 일반화 라그랑주

- ✓ 부등식 제약을 추가적으로 다루고자 함
- ✓ 하나의 부등식 제약 $g(\theta) \leq 0$ 이 있는 경우를 고려
- ✓ $\hat{L} = L(\theta) + \infty \mathbb{I}(g(\theta) > 0) \rightarrow$ 불연속 함수
- ✓ 벌점 가중치 역할을 하는 $\mu \geq 0, L(\theta, \mu) = L(\theta) + \mu g(\theta)$
- ✓ $\hat{\mathcal{L}}(\theta) = \max_{\mu \geq 0} L(\theta, \mu) = \begin{cases} \infty & \text{if } g(\theta) > 0, \\ \mathcal{L}(\theta) & \text{otherwise} \end{cases}$
- ✓ $\min_{\theta} \max_{\mu \geq 0, \lambda} L(\theta, \mu, \lambda)$
- ✓ $L(\theta, \mu, \lambda) = \mathcal{L}(\theta) + \sum_i \mu_i g_i(\theta) + \sum_j \lambda_j h_j(\theta)$

- KKT Conditions

- ✓ Primal Feasibility
 - 최적해 θ 자체가 제약을 만족해야 함
 - $g(\theta) \leq 0, h(\theta) = 0$
- ✓ Stationarity
 - 라그랑지안 함수의 도함수가 0이 되어야 함
 - $\nabla \mathcal{L}(\theta^*) + \sum_i \mu_i \nabla g_i(\theta^*) + \sum_j \lambda_j \nabla h_j(\theta^*) = 0$
- ✓ Dual Feasibility
 - 벌점의 관점에서 승수는 음수이면 안됨
 - $\mu_i \geq 0$
- ✓ Complementary Slackness
 - $\mu \odot g = 0$ (μ 와 g 둘 중 하나는 반드시 0 이어야 함)
 - Case 1 $\rightarrow g(\theta) = 0$, 해가 제약의 경계 위에 놓이며 사실상 등식 제약처럼 됨
 - Case 2 $\rightarrow g(\theta) < 0$, 최적해가 제약 조건의 경계선에 있지 않으므로 패널티인 μ 가 0이 됨

Q&A

감사합니다.